Detecting Mental Health Signals from Reddit Posts

Machine Learning Final Project Report

Harshitha Sai Maddukuri, Bhanu Pragnya Ravikiran, Kushagra Aggarwal
Khoury College Of Computer Sciences
Northeastern University
Boston, USA
Maddukuri.h@northeastern.edu, ravikiran.b@northeastern.edu, aggarwal.kush@northeastern.edu

***Abstract*— This report contains the motivation, implementation, and analysis of the final project for CS 6140 Machine Learning. The purpose of this report is to identify our problem, describe the process and results of our project, and comment on further related exploration. Our project concerns the application of machine learning classifier models to the problem of mental health detection from social media text. The models take Reddit posts as input, which are preprocessed, transformed into TF-IDF features, and then classified into mental health related categories using several classical models including logistic regression, Multinomial Naive Bayes, linear SVM, and Random Forest. The report is intended to provide an overview of our chosen task, an in-depth look into our implementation and application of machine learning models, and a discussion on the results of our implementation.**

1. INTRODUCTION & MOTIVATION

Mental health disorders such as depression, anxiety, adhd etc are increasingly prevalent and often underdiagnosed, mostly because people may be reluctant or unable to take medical support. At the same time, we also observed that many people openly express their thoughts and emotions on social media that indirectly reflect mental health conditions. We found that this problem is very relevant and important in today's world. So we developed a mental health detection system that reads social media posts and predicts if the individual suffers from any mental condition. In this system each post is labeled according to the underlying mental health status it suggests (e.g., anxiety, depression, neutral), and the task is then to classify future posts into these categories based

solely on their textual content. Classifying social media text may be difficult due to the overlapping and informal nature of language used across categories. The successful application of such a system could provide insights into how social media data can be leveraged for early mental health risk identification, potentially supporting outreach efforts and reducing the gap between those who need help and those who receive it.

2. DATASET & EXPLORATORY DATA ANALYSIS

We sourced our dataset of reddit posts labelled by mental health category from kaggle. Our dataset is a single csv file containing **5,957 posts** drawn from subreddits associated with depression, anxiety, and general discussion. Each post includes a title, a text body, and a numeric target label (0–4). In our project, we focus on three higher-level mental health states: **depression**, **anxiety**, and **neutral** so to obtain more interpretable labels and more examples per class, we remapped the original numeric targets into three broader categories : 0 and 4 were merged into *anxiety, 1* was mapped to *depression and* 2 and 3 to *neutral.* Then we performed dataset cleaning and preprocessing which yielded a cleaned dataset of 5,957 posts, each represented by its combined text and one of three class labels: anxiety, depression, or neutral. To better understand the data, we perform several exploratory data analysis (EDA) steps. First, we examined the **class distribution**, producing a bar chart that shows the number of posts in each of the three categories. Since we combined certain classes in anxiety and neutral, there is class imbalance between depression, anxiety and neutral which will be later handled in model training phase.

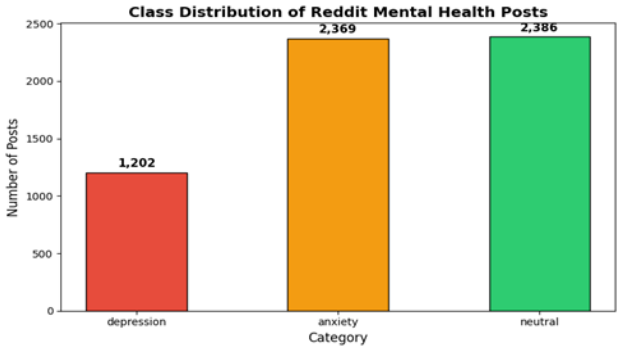

Fig.1. *Class distribution of Reddit mental health posts across three categories: depression (1,202), anxiety (2,369), and neutral (2,386).*

We then compute **post length statistics** by counting the number of words per post and summarizing the distribution of lengths within each class (mean, median, maximum), along with histograms of post length for anxiety, depression, and neutral posts.

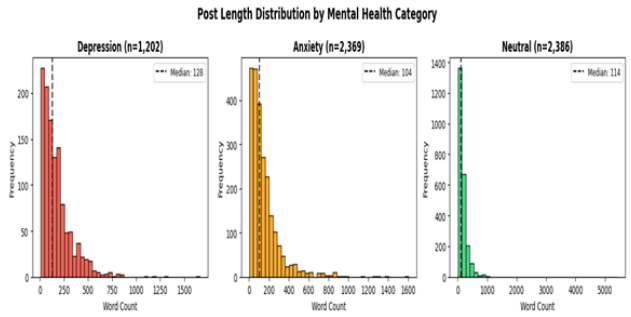

Fig. 2. *Post length distribution (word count) by mental health category. Dashed lines indicate the median for each class.*

The results showed clear differences by class:

- Depression posts had the longest median length (128 words), indicating that users describing depressive experiences often write more detailed, extended narratives.
- Neutral posts had a slightly shorter median (114 words) but the greatest variability, with lengths up to 5,419 words—likely reflecting long moderator announcements and informational content typical in broader mental health subreddits.
- Anxiety posts were the shortest on average, with a median of 104 words, aligning with anxiety-related writing that tends to focus on specific triggers or situations.

Overall, these patterns suggest that post length and writing style could provide a useful additional signal for classification, complementing the TF-IDF features used in the model.

Next, we explored the **vocabulary** used in each category. We cleaned the text by lowercasing, removing URLs, punctuation, digits, and stopwords, and then created **word clouds** for each class to visualize the most frequent terms. Then we further refined this analysis by extending the stopwords list with common, non-discriminative words and applying lemmatization, and regenerated word clouds and bar plots of the **top 10 words per class**.

Fig. 3. *Word clouds for depression, anxiety, and neutral posts after stopword removal and lemmatization, visualizing the most frequent terms per class.*

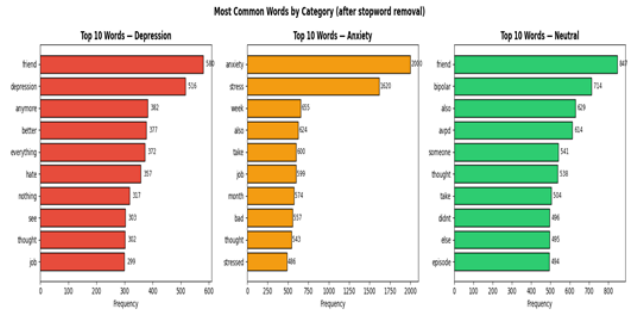

Fig. 4. *Top 10 most frequent words per mental health category after extended stopword removal, shown as horizontal bar plots.*

After applying the refined stopword list and lemmatization, the word clouds and top 10 word bar charts revealed clearly distinct vocabularies per class:

- Depression posts were characterized by words such as "friend", "anymore", "hate", and "depressed", reflecting themes of isolation, hopelessness, and emotional pain.
- Anxiety posts prominently featured "stress", "anxiety", "job", "panic", and "stressed", pointing to situational triggers and physical symptoms of anxiety.
- Neutral posts showed clinical terminology such as "bipolar", "avpd", "episode", and "diagnosed", consistent with their origin in general mental health discussion communities.

These distinct vocabularies confirm that the three classes carry meaningful linguistic differences, supporting the feasibility of text-based classification.

Finally, we computed a summary table of basic statistics (post counts, average, median, and maximum word counts) by labels and saved the cleaned dataset and EDA outputs for use in subsequent modeling.

```
=== EDA Summary Table ===
            total_posts  avg_word_count  median_word_count  max_word_count
label
anxiety            2369          148.82              104.0            1606
depression         1202          172.62              128.5            1663
neutral            2386          155.84              114.0            5419

Saved: eda_summary.csv
Saved: cleaned_reddit_data.csv
```

Fig. 5. *EDA summary statistics table showing total post count, average, median, and maximum word count per class label.*

For model evaluation, we split the cleaned dataset into training and test sets using an 80/20 stratified split on the three class labels (anxiety, depression, neutral). This gave us a training set used for fitting the TF–IDF vectorizers and models, and a test set that is used only for final evaluation. This ensures that the class proportions in the

test set closely match those in the full dataset, providing a more reliable estimate of real-world performance.

3. IMPLEMENTATION

All classifier models were implemented in Python using scikit-learn and can be found in our project repository. We used scikit-learn for model training and evaluation, NLTK for text preprocessing, and pandas and NumPy for data manipulation. After preprocessing, the text was converted to numerical features with a TF–IDF vectorizer that used sublinear term-frequency scaling, Unicode accent stripping, and a minimum document frequency of 2. Our baseline TF–IDF setup used up to 5,000 features and included both unigrams and bigrams.

A. Logistic Regression

We trained two Logistic Regression models: one with L2 regularization and one with L1 regularization, both using the liblinear solver and up to 2,000 iterations. L2 regularization shrinks all coefficients smoothly and helps prevent overfitting, while L1 regularization pushes many coefficients exactly to zero, effectively doing feature selection. By comparing these two versions, we can see whether letting the model automatically select a smaller set of important TF–IDF features (via L1) improves performance over the more standard L2 setup.

B. Multinomial Naive Bayes

We included a Multinomial Naive Bayes classifier as a simple, fast baseline. This model estimates how likely each word (or n-gram) is under each class and then uses Bayes' rule to pick the most likely class for a new post. It assumes features follow a multinomial distribution, which fits common text representations like counts and TF–IDF reasonably well, and it trains very quickly even on high dimensional text data.

C. Linear SVM:

We trained a Linear Support Vector Machine using a one-vs-rest strategy, meaning the model learns a separate decision boundary for each class against all others. Rather than estimating probabilities, it works by finding the hyperplane that best separates classes in the feature space while maximizing the margin between them. For high-dimensional text data, this turns out to be a particularly good fit. TF-IDF vectors are sparse and large, and Linear SVM tends to handle that kind of input efficiently without needing much tuning. The regularization parameter C controls the trade-off between fitting the training data tightly and keeping the decision boundary smooth. We started with the default setting to see how a strong linear classifier compares to the ensemble approach.

D.Random Forest (300 trees):

We trained a Random Forest classifier with 300 decision trees, each built on a random subset of the training samples and the features that were available. The final prediction is made by majority vote across all trees. The randomness that was added during training is what makes this work; since each tree sees a slightly different view of the data, the ensemble as a whole tends to be more robust. and less prone to overfitting than any individual tree would be on its own. It also captures non-linear relationships between features naturally, which is something linear models can't do. Given that posts from different subreddits likely carry different topic-level patterns and vocabulary combinations, the ability to model feature interactions rather than just individual word weights felt worth exploring here.

4. EVALUATION METHODOLOGY AND RESULTS:

To keep things systematic, all five models were tested across every one of the 12 TF-IDF configurations, which gave us 60 experimental combinations to work with. The main metric throughout was macro-averaged F1, chosen specifically because it treats each class equally, which matters here given the uneven class distribution. Accuracy was tracked alongside it but played more of a supporting role.

TABLE I. Test Results

Model	Accuracy	Macro F1	Marco Precision	Macro Recall
Random Forest	0.8398	0.8214	0.8571	0.8053
Linear SVM	0.8314	0.8171	0.8250	0.8113
LogReg L2	0.7987	0.7736	0.8063	0.7601
LogReg L1	0.7903	0.7713	0.7861	0.7626
Naive Bayes	0.7416	0.6673	0.7764	0.6647

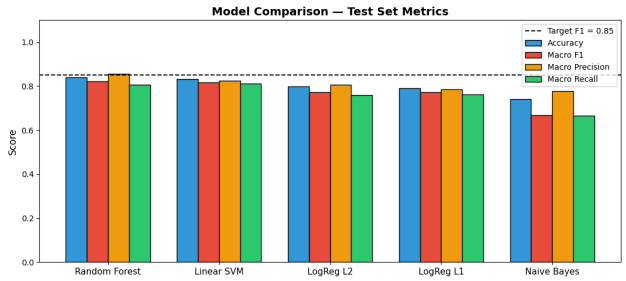

Fig. 6. *Model comparison on the test set across four metrics: accuracy, macro F1, macro precision, and macro recall for all five classifiers. The dashed line indicates the target F1 of 0.85.*

From these results, **Random Forest performed the best**, with a macro F1 score of **0.8214** and accuracy of about **84%**. **Linear SVM** was very close behind. Both logistic regression models gave similar performance around **0.77 F1**, while **Naive Bayes performed the worst**.

Cross-Validation Results:

To make sure our results are reliable and not just from one random data split, we used stratified 5-fold cross-validation on the full dataset (5,957 posts).

We also made sure to train the TF-IDF vectorizer separately inside each fold instead of training it once before splitting the data. This avoids data leakage and gives a fair evaluation.

TABLE II. *Stratified 5-fold cross-validation results showing mean and standard deviation of accuracy and macro F1 for all five models.*

Model	CV Accuracy (mean)	CV Accuracy (std)	CV F1 Macro (mean)	CV F1 Macro (std)
LogReg L2	0.8214	0.0062	0.7933	0.0079
LogReg L1	0.8001	0.006	0.7775	0.0084
Naive Bayes	0.763	0.0081	0.6964	0.0132
Linear SVM	0.8414	0.0058	0.8241	0.0063
Random Forest	0.8469	0.0065	0.8244	0.0077

The results show that Random Forest and Linear SVM perform almost the same and give the best results. The cross-validation scores are very close to the test results, which means the models are stable and not overfitting.

Also, the standard deviation values are small, which means the performance is consistent across different data splits.

Ablation Study:

We also ran an ablation study to test different TF-IDF settings. In total, we tested 60 combinations (12 TF-IDF configurations × 5 models).

We varied:

Number of features: 500, 1k, 5k, 10k
N-grams: unigrams, bigrams, and both

TABLE III. *Top 10 TF-IDF configurations ranked by macro F1 score across the full ablation study.*

TF-IDF Name	Max Features	N-gram Range	Min DF	Model	Accuracy	Macro F1
uni_bi_1k	1000	(1, 2)	2	rf	0.8498	**0.8371**
uni_bi_500	500	(1, 2)	2	rf	0.8482	0.8355
uni_1k	1000	(1, 1)	2	rf	0.8456	0.8317
uni_500	500	(1, 1)	2	rf	0.8473	0.8310
uni_bi_5k	5000	(1, 2)	2	rf	0.8398	0.8214
uni_bi_5k	5000	(1, 2)	2	linear_svm	0.8314	0.8171
uni_10k	10000	(1, 1)	2	linear_svm	0.8314	0.8151
uni_5k	5000	(1, 1)	2	rf	0.8322	0.8121
uni_bi_10k	10000	(1, 2)	2	linear_svm	0.8263	0.8107
uni_5k	5000	(1, 1)	2	linear_svm	0.8272	0.8103

One interesting result is that Random Forest performed the best in most cases. We were surprised that linear models like SVM didn't work better for text data. Random Forest worked best with smaller feature sizes (500–1,000). This could be because the model can find useful patterns more easily with fewer features.

We also saw that unigrams + bigrams gave the best results, unigrams alone worked reasonably well, and only bigrams worked the worst. This means that unigrams are important and bigrams are only helpful when used with them.

For more analysis, we looked at model behavior through confusion matrices, ROC curves, learning curves, and a closer look at where things went wrong.

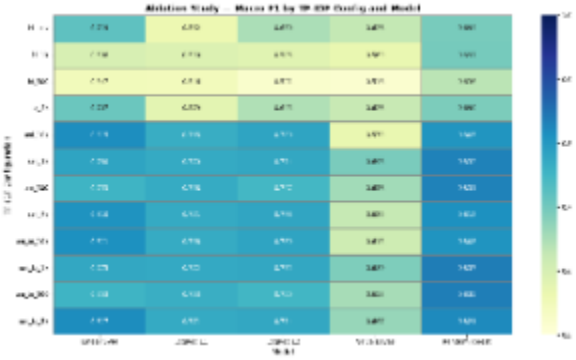

Fig. 7. *Ablation study heatmap showing macro F1 scores across 60 experimental configurations (12 TF-IDF settings × 5 models), sorted by F1 score.*

So, unigrams are important, and bigrams are helpful only when used together with them.

Additional Analysis:
Beyond the core metrics, we dug into model behavior through confusion matrices, ROC curves, learning curves, and a closer look at where things went wrong.

Confusion Matrices:
All models showed a similar pattern. Posts about depression and anxiety were often predicted as neutral. Neutral posts were the easiest to classify correctly.

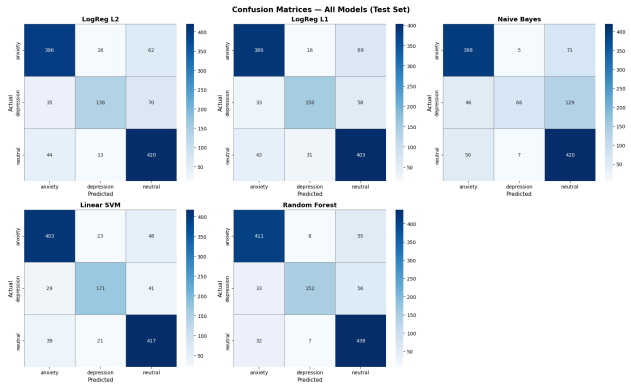

Fig. 8. *Confusion matrices for all five models on the test set, showing predicted vs. actual labels across anxiety, depression, and neutral classes.*

ROC Curves:
We used a one-vs-rest approach, meaning we created one ROC curve for each class in each model (15 curves in total). Since Linear SVM does not give probability outputs, we normalized its scores before calculating AUC. Random Forest achieved the highest overall AUC.

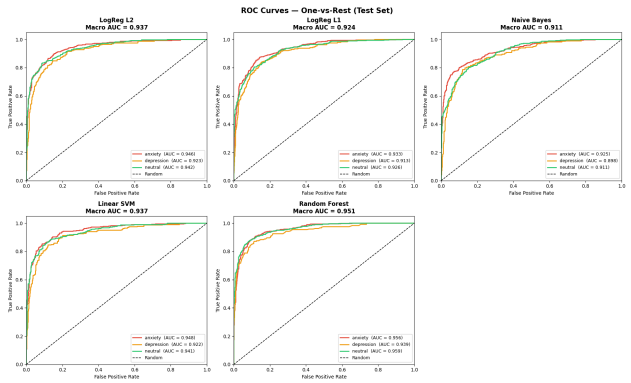

Fig. 9. *ROC curves using a one-vs-rest strategy for all five models on the test set. Macro AUC values are shown per model.*

Learning Curves:
We trained the models using different amounts of data, from 10% up to 100%. For the best models, the validation F1 score kept improving as we added more data. The difference between training and validation scores stayed small, which shows the models are not overfitting. This also means they could perform even better if we had more data.

Since the gap between training and validation is small at all stages, it suggests low variance. At the same time, because performance keeps improving with more data, it points to slight underfitting. In simple terms, both Random Forest and Linear SVM can still learn more and would likely improve further with a larger dataset.

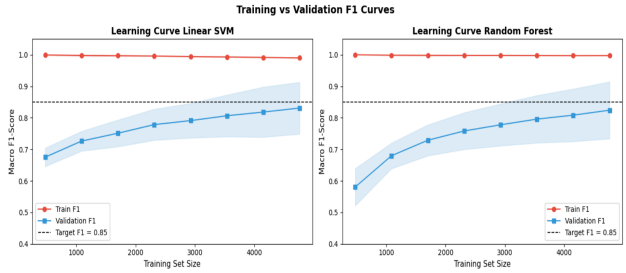

Fig. 10. *Learning curves showing training vs. validation macro F1 as a function of training set size for Linear SVM and Random Forest.*

Error Analysis:
Out of 1,192 test posts, the model got 191 wrong, roughly 16%. Most of those mistakes followed the same pattern: depression and anxiety posts written in a calm, practical tone got quietly filed away as neutral because nothing in the wording set off any alarm bells.

The flip side to a neutral post venting about an embarrassing situation got flagged as anxiety simply because the words looked familiar. Depression getting mistaken for anxiety was the third most common error, mostly in posts where someone described physical symptoms like poor sleep or restlessness rather than emotional distress. The model isn't doing anything wrong per se it's just working with word counts, and word counts can only take you so far when the same words mean very different things depending on how they're used.

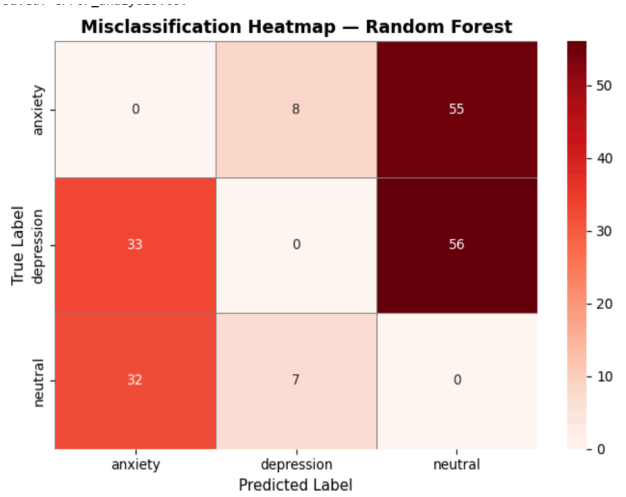

Fig. 11. *Misclassification heatmap for the best-performing model (Random Forest), showing the count of each error type across true and predicted label pairs.*

5. RESULTS AND CONCLUSIONS

The best model in our experiments was Random Forest. The basic setup got an accuracy of 83.98% and a macro F1 score of 0.8214 on the test set. After we changed the TF-IDF settings, we found a slightly better setup using 1,000 features with both unigrams and bigrams. This improved the results to 84.98% accuracy and 0.8371 macro F1.

When we ran cross-validation, the macro F1 stayed around 0.8244 ± 0.0077. This shows that the model is stable and the results are not dependent on just one train-test split.

One interesting result was that Random Forest did better than Linear SVM when the feature size was small, which is a bit unexpected for text data. Usually, Linear SVM works better for high-dimensional data.

Still, both Random Forest and SVM performed much better than Logistic Regression and Naive Bayes in our experiments.

We also observed that using unigrams + bigrams gave the best results, especially with smaller feature sizes. Models that used only bigrams performed poorly in all cases.

Finally, we looked at the errors made by the best model. Out of 1,192 test posts, the model made 191 mistakes, which is about 16%. The breakdown is:

TABLE IV. *Breakdown of the 191 misclassifications made by the best-performing Random Forest model on the 1,192 test posts, grouped by true label and predicted label pair.*

True Label	Predicted Label	Count
Depression	Neutral	56
Anxiety	Neutral	55
Depression	Anxiety	33
Neutral	Anxiety	32
Anxiety	Depression	8
Neutral	Depression	7

Most of the mistakes were cases where depression or anxiety posts were predicted as neutral. Together, these made up more than half of all errors. When we looked at some examples, this made sense.

For example, some depression posts included words related to self-improvement, like yoga, meditation, or sleep routines. These words also appear in neutral posts, so the model got confused and labeled them as neutral. In other cases, the posts were very short, so there wasn't enough information for the model to make the right prediction.

- We also saw some neutral posts that used strong emotional words, which caused the model to label them as anxiety.
- Overall, the main problem is that TF-IDF does not understand context. It treats each word separately, so it cannot tell the difference between "wellness" language and "coping" language when the same words are used.

6. FUTURE WORK:

More training data: Given that validation F1 has not plateaued at maximum training size, collecting more data is the highest priority next step. The depression class in particular is likely too small for stable generalization. Expanding additional Reddit datasets or combining with Twitter and forum data would directly address this bottleneck and test whether the 0.85 target is achievable with classical methods at scale.

Transformer-based models: Replacing TF-IDF with a contextual language model such as BERT or MentalBERT a variant pretrained specifically on mental health corpora would directly address the tone-blindness identified in the error analysis. These models encode sentence-level semantics rather than treating words as independent signals, which would help classify the understated and action-oriented posts that currently fall through.

Enriched features: Handcrafted features such as post length, punctuation density, sentence fragmentation score, and VADER sentiment scores could capture writing-style differences observed in the EDA but invisible to TF-IDF. These differences were clearly present: depression posts are longer; anxiety posts more fragmented but were not incorporated into the models.

Subreddit metadata: Including the source subreddit as a categorical feature would be a high-signal addition, since the community a post originates from carries strong prior probability about its class.

Calibrated confidence scores: For any real-world deployment scenario, Platt scaling or isotonic regression could be applied post-hoc to ensure that model confidence scores are reliable enough to support decision-making in a flagging or support system.

7.WORKS CITED :

[1] Tadesse, M. M., Lin, H., Xu, B., & Yang, L. (2019). Detection of Depression-Related Posts in Reddit Social Media Forum. IEEE Access, 7, 44883–44893.

[2] Coppersmith, G., Dredze, M., & Harman, C. (2014). Quantifying Mental Health Signals on Twitter. ACL Workshop on Computational Linguistics and Clinical Psychology.

[3] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of rMachine Learning Research, 12, 2825–2830.

[4] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

[5] Chawla, N. V., et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321–357.